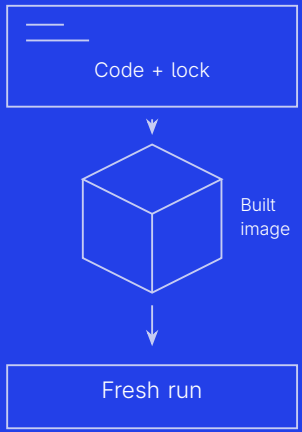
WEEK 2 / LAB 03

MLOps: Machine Learning Operations

Same code.
Clean environment.

Dependency locking, Docker build
& clean-environment rerun

Continue your team's Bike Demand project.



LEARNING OUTCOMES

Make the rerun verifiable

01	Lock	Check that dependencies match the committed lock.
02	Package	Build an image without copying the host environment.
03	Verify	Rerun the same baseline; explain the observed differences.

Package the baseline; do not redesign it

Continue from the reviewed Lab 2 code in the same team repository.

KEEP FIXED

Data and chronological split
Features and baseline settings
Training and validation behaviour

CHANGE HERE

Environment checks
Docker packaging
Clean-run instructions and evidence

No API, cloud deployment or test-set evaluation in Lab 3.

PART 1 / LOCK

Freeze the inputs before the rerun.

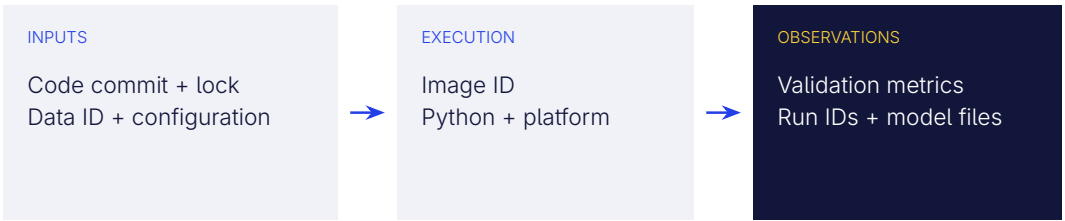
A lock is one part of the experiment's identity.

THE NEXT THREE CHECKS

Identify the inputs.
Validate the lock.
Check the runtime.

Trace inputs to the observed result

Link what you ran to the resulting metrics, model and run IDs.



Record actual identifiers; a screenshot alone does not connect them.

The manifest declares; the lock resolves

Use both committed files, then validate before installing.

PYPROJECT.TOML

Project intent

Python compatibility
Dependency constraints
Selected groups and project metadata

UV.LOCK

Resolved dependency choices

Direct and transitive packages
Versions and distribution information
Platform-dependent resolution

A lock does not freeze your data, host kernel or model results.

A stale lock must stop setup, not change silently

Run from the supplied project root, using the approved Python and uv.

PREFLIGHT COMMANDS

```
git status --short
git rev-parse HEAD
uv lock --check
uv sync --locked
uv run --locked python --version
docker version
```

INTERPRET THE CHECK

`--locked` verifies.

A mismatch is an error, not permission to replace the lock.

`--frozen` skips the freshness check; it is not the repair.

Missing lock, daemon or approved package? Record the blocker and stop.

TEAM ACTIVITY / LOCK AUDIT

Can another member identify your inputs?

Find the commit, lock, data manifest and baseline configuration.

Run preflight; record the real outcome.
Keep missing or blocked checks visible.

SUGGESTED TIME

10:00

ROTATE TASK ROLES

Driver checks.
Reviewer cross-checks.
Recorder captures evidence.

Deliverable: an input inventory and check status.

PART 2 / PACKAGE

Build the runtime, not your laptop.

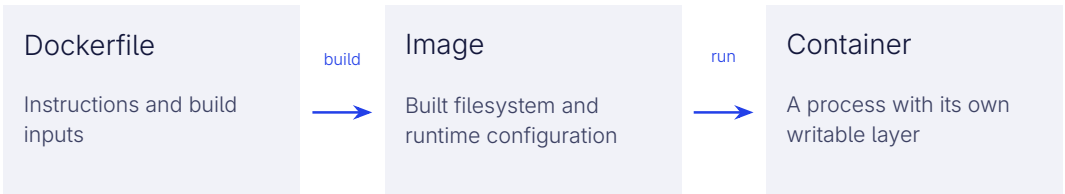
Start from the supplied Docker template and the committed lock.

THE NEXT THREE CHECKS

Control the context.
Install from the lock.
Run checks in the image.

Build a recipe; run its image

Build once, then start fresh containers from that recorded image.



An image packages a runtime; it does not replace the host kernel.

Control what enters the build context

The final dot in the build command selects the project context.

INCLUDE INTENTIONALLY

Code, tests and package metadata
Committed lock and baseline config
Approved data snapshot + attribution

EXCLUDE WITH .DOCKERIGNORE

`.venv`, caches and `.git`
Generated artifacts and tracking state
`.env` files and credentials

Inspect both the context and COPY rules; keep required inputs available.

Install from the committed lock

Complete the supplied Dockerfile; keep its approved runtime settings.

DOCKERFILE EXCERPT

```
COPY pyproject.toml uv.lock ./
COPY README.md ./
COPY src ./src
COPY tests ./tests
RUN uv sync --locked --no-editable
```

CHECK THE FULL TEMPLATE

Pinned base + uv version
Correct working directory
Data/config COPY rules

Keep test dependencies for this lab; do not add `--no-dev`.

No copied host environment and no unreviewed dependency upgrade.

A build still needs an execution check

Commit packaging changes; build from a clean project checkout.

BUILD AND TEST COMMANDS

```
git status --short
git rev-parse HEAD
docker build -t bike-demand:lab03 .
docker image inspect --format '{{.Id}}' bike-demand:lab03
docker run --rm --network none \
  --entrypoint .venv/bin/python bike-demand:lab03 \
  -m pytest -q
```

Run from a clean checkout; do not mount host source or environments.

Save the build log, image ID, test result and checked code revision.

Export outputs before removing the container

A fresh container should not silently reuse a previous model or tracking database.

NEW TRAINING CONTAINER

Same image and input files
Fresh writable output directories
New metrics, model and local run IDs



RETAIN EVIDENCE

Keep the stopped container.
Export results and readback.
Remove only after capture.

Do not use –rm for a training run whose outputs you still need.

PART 3 / VERIFY

Run it clean. Explain the result.

A teammate should be able to follow your evidence, not your memory.

THE NEXT THREE CHECKS

Rerun unchanged inputs.
Compare observations.
Review and hand over.

Rerun the baseline without host state

Use the built image; keep the named container so its outputs can be exported.

FRESH RUN COMMAND

```
docker run --name lab03-rerun-a --network none \
  --entrypoint .venv/bin/python bike-demand:lab03 \
  -m bike_demand.train --config configs/baseline.yaml
```

01

Capture logs and export run A.

02

Repeat as lab03-rerun-b.

03

Compare A and B with Lab 2.

No host mounts, no previous outputs, no hyperparameter changes.

Compare inputs before comparing MAE

Use the same validation rows for the naive predictor and Random Forest.

CHECK	RECORD AND INTERPRET
Inputs and environment	Commit, data ID, config, lock and image/platform
Validation observations	Both MAEs, row counts and finite predictions
Saved outputs and identity	New run IDs; model metadata and reload/readback

Report observed differences; a new run ID does not mean new inputs.

Find the first failing boundary

A useful diagnosis names the failed step, its message and the next check.

OBSERVED FAILURE	NEXT CHECK
Lock validation stops	Check approved manifest/lock and tool versions.
Build or imports fail	Inspect COPY context, package files and platform.
Training fails or MAE differs	Read the first error; compare data/config and outputs.

Do not fix setup by hiding tests, replacing data or weakening lock checks.

FAILURE CHALLENGE / DISPOSABLE COPY

A missing lock must fail visibly.

Use the supplied isolated exercise copy without `uv.lock`.

Run `uv lock –check`.

Capture the error. Restore the approved lock; recheck without replacing its bytes.

SUGGESTED TIME

10:00

EXPLAIN TO A TEAMMATE

What failed?
Why is that useful?
What proves recovery?

Do not delete the shared lock or commit the exercise fault.

Review the current revision

A different teammate reviews the diff and follows the rerun instructions.

01	Inspect the packaging	Check ignore/COPY rules, approved runtime and locked sync.
02	Follow the execution	Locate image identity, actual logs and exported run results.
03	Resolve before merge	Rerun relevant checks after changes; record any blocker.

The team merges after review and current checks; local tests are not remote CI.

Hand over the package and your evidence

Keep team implementation and personal reflection in their existing repositories.

TEAM PROJECT REPOSITORY

Reviewed package and run evidence
Trusted model + matching metadata export
Artifact readback instructions + next owner

PERSONAL REPORT REPOSITORY

One `reports/week-2.md` for Labs 3–4
Commit directly to personal `main`
No report PR, CI, tag or Release

After Lab 4: ungraded team checkpoint m2; Week 2 individual report is 10%.

EXIT CHECK / SHOW, DO NOT ASSUME

What can your teammate verify?

01	Locked	The approved manifest and lock pass validation.
02	Packaged	The recorded image runs checks without host state.
03	Compared	Fresh-run inputs and observed results can be traced.

Next / Lab 4: Prediction API, Input Validation & Containerised Tests